❷ Name of the circuit breaker

❸ Forwards request to this URI when the circuit is open

❹ No fallback defined for this circuit breaker.

The next step is configuring the circuit breaker.

9.2.2 Configuring a circuit breaker with Resilience4J

After defining which routes you want to apply the `CircuitBreaker` filter to, you need to configure the circuit breakers themselves. As often in Spring Boot, you have two main choices. You can configure circuit breakers through the properties provided by Resilience4J or via a `Customizer` bean. Since we're using the reactive version of Resilience4J, the specific configuration bean would be of type `Customizer<ReactiveResilience4JCircuitBreakerFactory>`.

Either way, you can choose to define a specific configuration for each circuit breaker you used in your application.yml file (`catalogCircuitBreaker` and `orderCircuitBreaker` in our case) or declare some defaults that will be applied to all of them.

For the current example, we can define circuit breakers to consider a window of 20 calls (`slidingWindowSize`). Each new call will make the window move, dropping the oldest registered call. When at least 50% of the calls in the window have produced an error (`failureRateThreshold`), the circuit breaker will trip, and the circuit will enter the open state. After 15 seconds (`waitDurationInOpenState`), the circuit will be allowed to transition to a half-open state in which 5 calls are permitted